

Testes em Telemetria de Fórmula 1 com PyTest e Coverage

Relatório final do trabalho prático: testes de caixa preta, cobertura estrutural e identificação de defeitos.

ALUNO	Luiz Gustavo Mella Massing
REPOSITÓRIO	github.com/LuizGustavoMM/UniSenai/tree/main/qualidade-e-teste-de-software/teste-F1
MÓDULO SOB TESTE	<code>estrategia_f1.py</code>
SUÍTE DE TESTES	<code>test_estrategia.py</code>
STACK	Python 3.11.6 · pytest 8.3.3 · pytest-cov 5.0.0
RESULTADO	81 execuções aprovadas · 100% de cobertura de instruções e de ramos · 3 defeitos identificados e corrigidos

1. Objetivo e escopo

Este trabalho implementa uma suíte de testes automatizados para o módulo de estratégia de corrida de uma equipe de Fórmula 1, responsável por determinar a volta de parada no pit stop e o composto de pneu ideal a partir dos dados de telemetria.

O trabalho foi conduzido nas três etapas previstas no enunciado: elaboração de casos de teste de caixa preta derivados exclusivamente da especificação; verificação da cobertura estrutural com `pytest-cov` até atingir 100% de instruções e de ramos; e execução da suíte contra o código base original para identificar e documentar os três defeitos propositais nele plantados.

Regras de negócio implementadas

REGRA	DEFINIÇÃO
Validação de entradas	<code>voltas_totais</code> deve estar entre 30 e 80 (inclusive) e <code>temp_asfalto</code> entre 10 °C e 60 °C (inclusive). Fora desses limites, a função lança <code>ValueError</code> .
Pneu Wet	Probabilidade de chuva $\geq 50\%$, independentemente da temperatura.
Pneu Soft	Sem chuva ($< 50\%$), temperatura do asfalto < 25 °C e desgaste do pneu $\geq 70\%$.
Pneu Hard	Temperatura ≥ 25 °C ou mais de 30 voltas restantes até o fim da corrida.

REGRA	DEFINIÇÃO
Alerta de emergência	Desgaste $\geq 80\%$ dispara alerta de parada obrigatória na próxima volta.

PRECEDÊNCIA E RETORNO

As regras de composto são avaliadas na ordem Wet $\rightarrow$ Soft $\rightarrow$ Hard, com retorno padrão Soft quando nenhuma delas se aplica (pista fria, stint curto e pneu íntegro). A função retorna um dicionário com as chaves `volta_pit_stop`, `composto`, `alerta_emergencia` e `mensagem`. A janela de parada é proporcional à vida útil restante do pneu, e as voltas restantes usadas na regra do Hard são `voltas_totais - volta_pit_stop`.

2. Mapeamento dos casos de teste de caixa preta

Os casos abaixo foram derivados **exclusivamente da especificação**, sem consultar a implementação, aplicando *particionamento de equivalência* (PE) e *análise de valor limite* (AVL). A suíte reúne 24 casos de caixa preta que, por conta dos `@pytest.mark.parametrize`, se expandem em 66 execuções.

R1. Validação de entradas

CP	TÉCNICA	ENTRADA	RESULTADO ESPERADO
CP01	AVL	<code>voltas_totais</code> $\in \{30, 31, 55, 79, 80\}$	aceita, limites são inclusivos
CP02	AVL / PE	<code>voltas_totais</code> $\in \{29, 0, -1, 81, 100\}$	ValueError
CP03	AVL	<code>temp_asfalto</code> $\in \{10, 11, 35, 59, 60\}$	aceita, limites são inclusivos
CP04	AVL / PE	<code>temp_asfalto</code> $\in \{9, 0, -5, 61, 120\}$	ValueError
CP05	robustez	<code>voltas_totais</code> = 20	mensagem identifica <code>voltas_totais</code>
CP06	robustez	<code>temp_asfalto</code> = 5	mensagem identifica <code>temp_asfalto</code>
CP07	robustez	dois parâmetros inválidos	ValueError, sem resultado parcial

R2. Escolha de pneu de chuva (Wet)

CP	TÉCNICA	ENTRADA	RESULTADO ESPERADO
CP08	AVL	<code>chuva</code> $\in \{50, 51, 75, 99, 100\}$	Wet, pois 50% já é chuva
CP09	PE	<code>chuva</code> = 80, <code>temp</code> $\in \{10, 24, 25, 40, 60\}$	Wet em todas as temperaturas
CP10	PE	<code>chuva</code> = 60, <code>desgaste</code> $\in \{0, 50, 70, 85, 100\}$	Wet em todos os desgastes
CP11	AVL	<code>chuva</code> = 49	não é Wet, ainda é pista seca

R3. Escolha de pneu macio (Soft)

CP	TÉCNICA	ENTRADA	RESULTADO ESPERADO
CP12	AVL	temp = 20, desgaste ∈ {70, 71, 90, 100}	Soft, pois 70% já é desgaste alto
CP13	AVL	desgaste = 75, temp ∈ {10, 15, 24}	Soft, pois 24 °C ainda é pista fria
CP14	PE	temp = 30, desgaste = 75	não é Soft, a condição é conjunta
CP15	PE	80 voltas, temp = 20, desgaste = 50	Hard, a condição é conjunta

R4. Escolha de pneu duro (Hard)

CP	TÉCNICA	ENTRADA	RESULTADO ESPERADO
CP16	AVL	temp ∈ {25, 26, 40, 60}	Hard, pois 25 °C já é pista quente
CP17	PE	80 voltas, temp = 15, desgaste = 45	Hard, mais de 30 voltas restantes
CP18	PE	40 voltas, temp = 20, desgaste = 50	Soft, nenhuma condição do Hard

R5. Alerta de parada de emergência

CP	TÉCNICA	ENTRADA	RESULTADO ESPERADO
CP19	AVL	desgaste ∈ {80, 81, 95, 100}	alerta True e mensagem obrigatória
CP20	AVL	desgaste ∈ {0, 40, 78, 79}	alerta False
CP21	funcional	desgaste = 40	mensagem informa a volta programada

R6. Contrato de retorno

CP	TÉCNICA	ENTRADA	RESULTADO ESPERADO
CP22	contrato	entrada válida	retorno traz as quatro chaves previstas
CP23	contrato	60 voltas	$1 \leq \text{volta_pit_stop} \leq 60$
CP24	funcional	desgaste 70 contra 20	desgaste maior antecipa a parada

3. Testes de caixa branca e cobertura de código

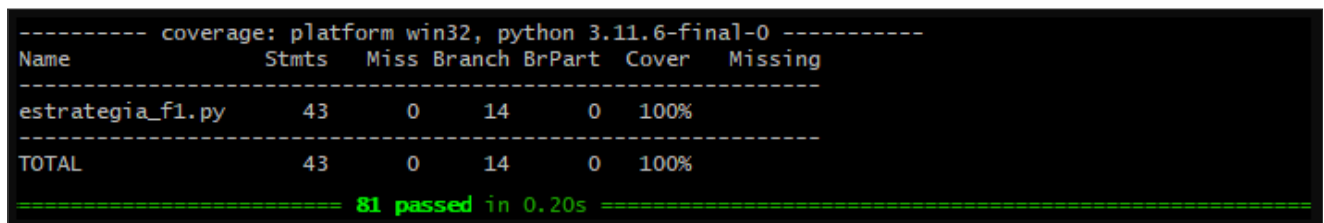
Concluída a Etapa 1, a estrutura interna do módulo foi analisada e 15 casos estruturais foram acrescentados à suíte. Enquanto os testes de caixa preta exercitam a função pública de ponta a ponta, estes atacam cada função auxiliar isoladamente, garantindo que todo ramo seja tomado nos dois sentidos e que cada sub-condição dos operadores lógicos seja, sozinha, responsável por um resultado observado.

CP	FUNÇÃO EXERCITADA	RAMO OU CAMINHO COBERTO
CP25	validar_entradas	ambos os if falsos, caminho feliz
CP26	validar_entradas	1ª sub-condição do 1º if, voltas < 30
CP27	validar_entradas	2ª sub-condição do 1º if, voltas > 80
CP28	validar_entradas	1ª sub-condição do 2º if, temp < 10
CP29	validar_entradas	2ª sub-condição do 2º if, temp > 60
CP30	calcular_volta_parada	if volta < 1 falso
CP31	calcular_volta_parada	if volta < 1 verdadeiro, saturação
CP32	calcular_volta_parada	arredondamento da janela fracionária
CP33	escolher_composto	caminho 1, retorno Wet
CP34	escolher_composto	caminho 2, retorno Soft
CP35	escolher_composto	caminho 3 pela 1ª sub-condição, temperatura
CP36	escolher_composto	caminho 3 pela 2ª sub-condição, voltas restantes
CP37	escolher_composto	caminho 4, retorno final por padrão
CP38	verificar_alerta_emergencia	resultado True no limite, 80%
CP39	verificar_alerta_emergencia	resultado False abaixo do limite, 79%

Captura da cobertura obtida

A cobertura de ramos está habilitada no `.coveragerc`, de modo que o comando abaixo reporta instruções e ramos simultaneamente.

```
pytest --cov=estrategia_f1 --cov-report=term-missing
```



```

----- coverage: platform win32, python 3.11.6-final-0 -----
Name                Stmts   Miss Branch BrPart  Cover   Missing
-----
estrategia_f1.py      43      0      14      0    100%
TOTAL                 43      0      14      0    100%
===== 81 passed in 0.20s =====

```

Figura 1. Saída do `pytest-cov` no terminal. 43 instruções e 14 ramos, nenhum deles descoberto: `Miss = 0`, `BrPart = 0`, `Cover = 100%`, com as 81 execuções aprovadas.

MÉTRICA	TOTAL	NÃO COBERTO	COBERTURA
Instruções (<i>statement coverage</i>)	43	0	100%
Ramos (<i>branch coverage</i>)	14	0	100%

```
===== 81 passed in 0.11s =====
```

Figura 2. Execução da suíte completa com `pytest -v` após as correções: 81 execuções, nenhuma falha.

4. Relatório dos bugs identificados no código base

A suíte desenvolvida foi executada contra o **código base original**. Das 81 execuções, 8 foram reprovadas, e elas se agrupam em **3 defeitos distintos** na lógica da aplicação. Cada defeito foi corrigido em um commit separado no repositório, o que permite rastrear defeito, teste e correção.

```
===== short test summary info =====
FAILED test_estrategia.py::TestValidacaoDeEntradas::test_cp01_voltas_dentro_do_intervalo_sao_aceitas[30] - ValueError: voltas_totais deve estar entre 30 e 80 (inclusive).
FAILED test_estrategia.py::TestValidacaoDeEntradas::test_cp01_voltas_dentro_do_intervalo_sao_aceitas[80] - ValueError: voltas_totais deve estar entre 30 e 80 (inclusive).
FAILED test_estrategia.py::TestEscolhaDePneuWet::test_cp08_chuva_igual_ou_acima_de_50_seleciona_wet[50] - AssertionError: assert 'Hard' == 'Wet'
FAILED test_estrategia.py::TestEscolhaDePneuSoft::test_cp14_temperatura_alta_com_desgaste_alto_nao_e_soft - AssertionError: assert 'Soft' != 'Soft'
FAILED test_estrategia.py::TestEscolhaDePneuSoft::test_cp15_temperatura_baixa_com_desgaste_baixo_e_stint_longo - ValueError: voltas_totais deve estar entre 30 e 80 (inclusive).
FAILED test_estrategia.py::TestEscolhaDePneuHard::test_cp17_stint_longo_seleciona_hard_mesmo_com_pista_fria - ValueError: voltas_totais deve estar entre 30 e 80 (inclusive).
FAILED test_estrategia.py::TestCaixaBrancaEscolherComposto::test_cp33_caminho_1_retorna_wet - AssertionError: assert 'Hard' == 'Wet'
FAILED test_estrategia.py::TestCaixaBrancaEscolherComposto::test_cp36_caminho_3_retorna_hard_por_voltas_restantes - AssertionError: assert 'Soft' == 'Hard'
===== 8 failed, 73 passed in 0.25s =====
```

Figura 3. Resumo da execução da suíte contra o código base original: 8 failed, 73 passed. Cada linha nomeia o caso de teste que detectou o defeito e o erro observado.

#	DEFEITO	FUNÇÃO AFETADA	CLASSIFICAÇÃO	TESTES QUE REVELARAM
1	Limites de voltas tratados como exclusivos	validar_entradas	erro de valor limite (off-by-one)	CP01 (30 e 80)
2	Limiar de chuva com comparação estrita	escolher_composto	operador relacional incorreto	CP08 (50), CP33
3	Condição do pneu Soft usando OU em vez de E	escolher_composto	operador lógico incorreto	CP14, CP15, CP17, CP36

Bug 1 · Limites de `voltas_totais` tratados como exclusivos

CÓDIGO COM DEFEITO

```
if voltas_totais <= VOLTAS_MINIMAS or voltas_totais >= VOLTAS_MAXIMAS:
    raise ValueError(...)
```

Descrição. A especificação define que o número total de voltas deve estar entre 30 e 80 **inclusive**. O código usava `<=` e `>=`, o que rejeita exatamente os dois valores de fronteira válidos. Uma corrida de 30 voltas ou de 80 voltas, ambas legítimas, era recusada com `ValueError`.

Deteção. O caso CP01 aplica análise de valor limite e verifica que 30 e 80 são aceitos. Duas das cinco execuções parametrizadas falharam:

```
CP01[30] - ValueError: voltas_totais deve estar entre 30 e 80 (inclusive).
CP01[80] - ValueError: voltas_totais deve estar entre 30 e 80 (inclusive).
```

O defeito produziu ainda falhas em cascata em CP15 e CP17, que usam corridas de 80 voltas e não chegavam sequer a avaliar a regra de composto.

Impacto. Bloqueio total do cálculo de estratégia em corridas nos limites do regulamento, com falso positivo de entrada inválida.

CORREÇÃO APLICADA

```
if voltas_totais < VOLTAS_MINIMAS or voltas_totais > VOLTAS_MAXIMAS:  
    raise ValueError(...)
```

Bug 2 · Limiar de chuva com comparação estrita

CÓDIGO COM DEFEITO

```
if porcentagem_chuva > LIMIAR_CHUVA_WET:  
    return PNEU_WET
```

Descrição. A regra determina que uma probabilidade de chuva **maior ou igual a 50%** selecione o pneu Wet. Com o operador `>`, uma probabilidade de exatamente 50% escapava da regra, caía nas condições de pista seca e retornava `Hard`.

Deteção. Duas frentes independentes apontaram o mesmo defeito: CP08, de caixa preta, com `chuva = 50`, e CP33, de caixa branca, que exercita o primeiro caminho de saída da função. Ambos reportaram `AssertionError: assert 'Hard' == 'Wet'`.

Impacto. Defeito crítico de segurança operacional. Em pista molhada no limiar da regra, o carro seria enviado à pista com composto de piso seco.

CORREÇÃO APLICADA

```
if porcentagem_chuva >= LIMIAR_CHUVA_WET:  
    return PNEU_WET
```

Bug 3 · Condição do pneu Soft usando OU em vez de E

CÓDIGO COM DEFEITO

```
if (temp_asfalto < LIMIAR_TEMPERATURA_SOFT  
    or desgaste_pneu >= LIMIAR_DESGASTE_SOFT):  
    return PNEU_SOFT
```

Descrição. A regra exige as **duas** condições simultaneamente: temperatura do asfalto inferior a 25 °C **e** desgaste do pneu maior ou igual a 70%. Com o operador `or`, qualquer uma das condições isolada já bastava, e o pneu Soft passava a ser escolhido em dois cenários errados:

- pista quente, com 25 °C ou mais, e pneu gasto, quando o correto seria `Hard`;
- pista fria com pneu praticamente novo e stint longo, quando o correto também seria `Hard`.

Como esse `if` antecede a regra do `Hard`, o retorno prematuro mascarava inteiramente a regra 3 nesses casos.

Deteção. Quatro casos reprovaram:

TESTE	CENÁRIO	ESPERADO	OBTIDO
CP14	temp = 30, desgaste = 75	diferente de Soft	Soft
CP15	80 voltas, temp = 20, desgaste = 50	Hard	Soft
CP17	80 voltas, temp = 15, desgaste = 45	Hard	Soft
CP36	voltas_restantes = 31, caixa branca	Hard	Soft

Impacto. Escolha de composto macio em condições de alta degradação térmica ou em stints longos, com desgaste acelerado e uma parada adicional não planejada na corrida.

CORREÇÃO APLICADA

```
if (temp_asfalto < LIMIAR_TEMPERATURA_SOFT
    and desgaste_pneu >= LIMIAR_DESGASTE_SOFT):
    return PNEU_SOFT
```

5. Conclusão

A suíte final reúne **39 casos de teste**, 24 de caixa preta e 15 de caixa branca, que se expandem em **81 execuções** por conta da parametrização. Todas passam contra o código corrigido, com cobertura de 100% tanto de instruções quanto de ramos.

Vale registrar um ponto de método observado durante o trabalho: os testes de caixa preta, sozinhos, já alcançariam cobertura de ramos elevada, mas foram os casos estruturais que isolaram cada sub-condição dos operadores lógicos. O Bug 3 ilustra bem o risco, o retorno prematuro do `Soft` mascarava inteiramente a regra do `Hard`, e sem CP36 a falha só apareceria de forma indireta. Cobertura alta, portanto, é condição necessária mas não suficiente, o que importa é qual condição foi responsável pelo resultado observado.

COMO REPRODUZIR

```
pip install -r requirements.txt
pytest -v
pytest --cov=estrategia_f1 --cov-report=term-missing
pytest --cov=estrategia_f1 --cov-report=html
```

O repositório contém ainda os documentos de apoio `README.md`, `MAPEAMENTO_TESTES.md` e `RELATORIO_BUGS.md`, além do histórico de commits que registra a evolução exigida pelo enunciado: código base com os defeitos, suíte de testes que os expõe, correções uma a uma e fechamento da cobertura em 100%.